

To begreper for feil

La $x_1, x_2, x_3, \dots$ være verdier fra en iterasjonsmetode for å finne en rot r av $f(x)$.

Vi skiller mellom to måletall for feilen

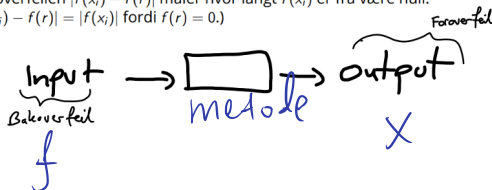
Definisjon (Foroverfeil)

Foroverfeilen $e_i = |x_i - r|$ viser hvor stor feilen er i den utregnede verdien x_i .
 ↑
 utregnet

↖ faktisk roten

Definisjon (Bakoverfeil)

Bakoverfeilen $|f(x_i) - f(r)|$ måler hvor langt $f(x_i)$ er fra være null.
 ($|f(x_i) - f(r)| = |f(x_i)|$ fordi $f(r) = 0$.)



Bakoverfeilen sier noe om hvor mye vi må **endre problemet** vi skal løse for at vår (numeriske) løsning blir en **eksakt løsning** på det endrede problemet.

Vi skal ved hjelp av halveringsmetoden løse ligningen

$$x^3 - 2x^2 + 4/3 x - 8/27 = 0$$

med minst 6 korrekte signifikante sifre.

Vi skal ved hjelp av halveringsmetoden løse ligningen

$$x^3 - 2x^2 + 4/3 x - 8/27 = 0$$

med minst 6 korrekte signifikante sifre.

Ligningen har en rot i $(0, 1)$,¹ og det skal være nok med 20 intervallhalvinger for å oppnå den ønskede nøyaktighet.²

i	a_i	$f(a_i)$	c_i	$f(c_i)$	b_i	$f(b_i)$
0	0.000000	—	0.500000	—	1.000000	+
1	0.500000	—	0.750000	+	1.000000	+
2	0.500000	—	0.625000	—	0.750000	+
3	0.625000	—	0.687500	+	0.750000	+
4	0.625000	—	0.656250	—	0.687500	+
5	0.656250	—	0.671875	+	0.687500	+
6	0.656250	—	0.664062	—	0.671875	+
7	0.664062	—	0.667968	+	0.671875	+
8	0.664062	—	0.666015	—	0.667968	+
9	0.666015	—	0.666922	+	0.667968	+
10	0.666015	—	0.666503	—	0.666922	+
11	0.666503	—	0.666748	+	0.666922	+
12	0.666503	—	0.666626	—	0.666748	+
13	0.666626	—	0.666687	+	0.666748	+
14	0.666626	—	0.666656	—	0.666687	+
15	0.666656	—	0.666671	+	0.666687	+
16	0.666656	—	0.666664	0	0.666671	+

men!
 $f(0.66664) = 0$

¹Ligningen har trippelroten $r = 2/3$.

²Av $|b - a|/2^{n+1} < 1/2 \times 10^{-6}$ fås at $n \geq 6/\log_{10} 2 \approx 19.9$.

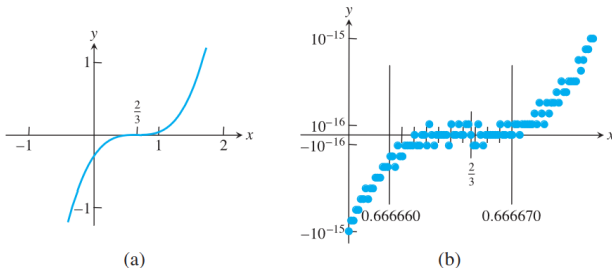


Figure 1.7 The shape of a function near a multiple root. (a) Plot of $f(x) = x^3 - 2x^2 + 4/3x - 8/27$. (b) Magnification of (a), near the root $r = 2/3$. There are many floating point numbers within 10^{-5} of $2/3$ that are roots as far as the computer is concerned. We know from calculus that $2/3$ is the only root.

```

1 from scipy import optimize
2 def f(x):
3     #naive polynomial evaluations
4     return x**3-2*x**2+4/3*x - 8/27
5 def g(x):
6     #horner's method for polynomial evaluations
7     return ((x-2)*x+4/3)*x-8/27
8
9 rootf=optimize.bisect(f,0,1)
10 rootg=optimize.bisect(g,0,1)

```

```

root found using naive polynomial evaluations: 0.6666641235351562
f(root) 0.0
root found using horner polynomial evaluations: 0.6666698455810547
g(root) 0.0

```

```

f(0.666662); -1.1102230246251565e-16
g(0.666662); -5.551115123125783e-17
f(0.666663); 0.0
g(0.666663); -5.551115123125783e-17
f(0.666664); 0.0
g(0.666664); -1.1102230246251565e-16
f(0.666665); 0.0
g(0.666665); 0.0

```

Merk at rundt $2/3$ ligger flyttallene **meget tett** med en avstand på 2^{-53} (så lenge vi er i $[1/2, 1)$).
 IEEE754 DP representasjonen $f(2/3)$ av $2/3$ er **eksakt** lik $6004799503160661/9007199254740992 =$
 $0.66666666666666662965923251249478198588 \dots$

Her er $BE = FE^3$ (Bakoverfeil $BE \approx 10^{-16}$ /Foroverfeil $FE \approx 10^{-5}$)

I tabellen er $f(x) = (x - 2/3)^3 = x^3 - 2x^2 + 4x/3 - 8/27$ evaluert for $x > 2/3$.

```

1 0.6666667666666666 0.0
2 0.6666668666666666 -1.1102230246251565e-16
3 0.6666669666666666 -1.1102230246251565e-16
4 0.6666670666666666 0.0
5 0.6666671666666666 0.0
6 0.6666672666666666 0.0
7 0.6666673666666666 0.0
8 0.6666674666666667 1.1102230246251565e-16
9 0.6666675666666666 -1.1102230246251565e-16
10 0.6666676666666667 0.0
11 0.6666677666666666 0.0
12 0.6666678666666667 0.0
13 0.6666679666666666 0.0
14 0.6666680666666667 1.1102230246251565e-16
15 0.6666681666666666 -1.1102230246251565e-16
16 0.6666682666666667 0.0
17 0.6666683666666666 -1.1102230246251565e-16
18 0.6666684666666667 0.0
19 0.6666685666666666 0.0
    
```

Merk funksjonsverdiene med **galt fortegn** (og 0.0)!

Dette skaper som sagt problemer ved bruk av halveringsmetoden (og andre løsningsmetoder som bygger på funksjonsevalueringer.)

Hva kan vi lese ut av eksemplet over?

Problemet er **ikke halveringsmetoden** (eller enn annen tilnæringsmetode som er avhengig av å beregne verdier av $f(x)$) da det er **begrenset hvor nøyaktig $f(x)$ kan beregnes i nærheten av roten r** . (Jfr. figuren på forrige lysark.)

Denne begrensningen gjelder alle metoder som kun har kjennskap til $f(x)$ under løsningen.¹

Her er bakoverfeilen i nærheten av $\epsilon_M = 2^{-52} \approx 2.2 \cdot 10^{-16}$ (for IEEE754 DP flyttall) mens foroverfeilen er i størrelsesorden 10^{-5} .

Og, da bakoverfeilen ikke kan reduseres ytterligere, kan heller ikke foroverfeilen det.²

Hvorfor gjelder for $f(x) = (x - 2/3)^3$ at $FE = \sqrt[3]{BE}$?

¹Vi har testet de avanserte ligningsløserne FindRoot og fzero fra henholdsvis Mathematica og MATLAB på neste lysark.

²Hvis vi ikke øker flyttallsnøyaktigheten da fra for eksempel IEEE754 double til IEEE754 long double.

Nedenfor viser resultatet av å løse ligningen $(x - 2/3)^2 = 0$ numerisk med henholdsvis Mathematica og MATLAB (det siste tatt fra SAUER, Ch. 1.3, s. 45) sine avanserte ligningslødere FindRoot og fzero:¹

```
1 f[x_] := (x - 2/3)^3 // Expand;
2 x // f // Print;
3 r1 = FindRoot[f[x], {x, 1}];
4 r2 = FindRoot[f[x], {x, 2}];
5 r1 // Print;
6 r2 // Print;
```

```
$ kernel
Mathematica 13.0.1 Kernel for Linux x86 (64-bit)
Copyright 1988-2022 Wolfram Research, Inc.

In[1]:= <<root.m
      8      4 x      2      3
      -(-) + --- - 2 x + x
      27      3
{x -> 0.666671}
{x -> 0.666669}
```

```
>> fzero('x.^3-2*x.^2+4*x/3-8/27',1)

ans =

0.66666250845989
```

¹Syntaks for disse funksjonene er heholdvis: Bestem en tilnærmelse til roten av ligningen i nærheten av 1 og 2, og 1.

Sensitivity Formula for Roots

Assume that r is a root of $f(x)$ and $r + \Delta r$ is a root of $f(x) + \epsilon g(x)$. Then

$$\Delta r \approx -\frac{\epsilon g(r)}{f'(r)}$$

if $\epsilon \ll f'(r)$.

Feilforstøringsfaktor.

Feilforstøringsfaktor=relativ foroverfeil/relativ bakoverfeil

$$\text{Feilforstøringsfaktor} = \left| \frac{\Delta r/r}{\epsilon} \right| = \frac{|g(r)|}{|r f'(r)|}.$$

Formlene ovenfor bevises ikke.

Formlene følger ved å bruke Taylor-utvikling om r for $f(r + \Delta r)$ og $g(r + \Delta r)$ der en tar med kun første ordens ledd. (Se SAUER 1.3.3, side 48.)

Eksempel 1.9 fra SAUER, s. 48

Estimate the largest root of $P(x) = (x-1)(x-2)(x-3)(x-4)(x-5)(x-6) - 10^{-6}x^7$.

Set $f(x) = (x-1)(x-2)(x-3)(x-4)(x-5)(x-6)$, $\epsilon = -10^{-6}$ and $g(x) = x^7$. Without the $\epsilon g(x)$ term, the largest root is $r = 6$. The question is, how far does the root move when we add the extra term?

The Sensitivity Formula yields

$$\Delta r \approx -\frac{\epsilon 6^7}{5!} = -2332.8\epsilon,$$

meaning that input errors of relative size ϵ in $f(x)$ are magnified by a factor of over 2000 into the output root. We estimate the largest root of $P(x)$ to be $r + \Delta r = 6 - 2332.8\epsilon = 6.0023328$. Using `fzero` on $P(x)$, we get the correct value 6.0023268. ◀

Vi ser at en feil i størrelseorden 10^{-6} i input (representasjonen av $f(x)$) forårsaker en feil i det tredje sifret i den beregnede roten.

$P(x) = (x-6) \times p(x)$, så $P'(x) = p(x) + (x-6) \times p'(x)$.

Så, $P'(6) = p(6) = 5!$

Dette gir at $\Delta r \approx -\epsilon g(6)/p'(6) = -\epsilon 6^7/5!$, som gitt ovenfor.

Wilkinson-polynomet

Wilkinson-polynomet:

$W(x) = (x - 20)(x - 19)(x - 18)(x - 17) \cdots (x - 2)(x - 1)$ har røtter lik 1, 2, 3, ..., 19, 20. Om vi ekspanderer polynomet får vi

$$\begin{aligned} W(x) = & x^{20} - 210 x^{19} + 20615 x^{18} - 1256850 x^{17} + 53327946 x^{16} \\ & - 1672280820 x^{15} + 40171771630 x^{14} - 756111184500 x^{13} \\ & + 11310276995381 x^{12} - 135585182899530 x^{11} + 1307535010540395 x^{10} \\ & - 10142299865511450 x^9 + 63030812099294896 x^8 \\ & - 311333643161390640 x^7 + 1206647803780373360 x^6 \\ & - 3599979517947607200 x^5 + 8037811822645051776 x^4 \\ & - 12870931245150988800 x^3 + 13803759753640704000 x^2 \\ & - 8752948036761600000 x + 2432902008176640000 \quad (1) \end{aligned}$$

Vi har for eksempel $W(20) = 0$, men hvilken verdi får vi i dobbel presisjon på en datamaskin?

$$W(20) \approx -2,7 \cdot 10^{10}$$

Hvorfor?

NB! Brukes Python til å beregne $W(20)$, så er det viktig at koeffisientene til polynomet er definert som **floats**. (Hvis ikke får vi "integer overflow".)

Feilen i Wilkinsonpolynomet

Feilen i koeffisientene til Wilkinson-polynomet når de representeres som flyttall med dobbel presisjon er

$$\begin{aligned}c_3 - fl(c_3) &= -512, \\c_4 - fl(c_4) &= 384, \\c_5 - fl(c_5) &= -160, \\c_6 - fl(c_6) &= 112, \text{ og} \\c_7 - fl(c_7) &= 16.\end{aligned}$$

I ekspandert form blir $W(x)$ lik

$$W(x) = 16x^7 - 112x^6 + 160x^5 - 384x^4 + 512x^3$$

på grunn av avrunding av koeffisientene.

Det er katastrofalt.

Merk at bidraget fra de andre polynomkoeffisientene i pertubasjonen $\epsilon g(x)$ av $W(x)$ er **neglisjerbart**.¹ (Det kan vises at denne pertubasjonen gir feilforsterkningsfaktoren $\approx 5 \times 10^7$ for roten $r = 20$, så denne roten er dårlig kondisjonert og forklarer datamaskinverdien på $W(20)$ fra forrige lysark. Vi dropper (de noe omstendelige) detaljene som kreves for å vise dette, men **detaljene er gitt på de neste tre lysarkene for de spesielt interesserte.**

¹ Disse koeffisientene er nøyaktig (eller eksakt) representert som flyttall i IEEE754 DP. I Python gir `int(cj) - int(float(cj))` differansene ovenfor. Ellers kan <https://www.binaryconvert.com> brukes (referert til tidligere).

IMAx2150: Begrenset nøyaktighet



Feilforsterknings/forstøringsfaktor

Eksempel for roten $r = 20$ for Wilkinson polynomet.

Feilforstøringsfaktor=relativ foroverfeil/relativ bakoverfeil

$$\text{Feilforstøringsfaktor} = \left| \frac{\Delta r/r}{\epsilon} \right| = \frac{|g(r)|}{|r f'(r)|}.$$

Eksempel (Wilkinson $r = 20$). $W'(20) = 19! \approx 1.2165 \cdot 10^{17}$
 $\epsilon g(20) \approx -2,7 \cdot 10^{10}$.

$$\Delta r \approx -\frac{-2,7 \cdot 10^{10}}{1.2165 \cdot 10^{17}} = 2.2355 \cdot 10^{-07}$$

$$\text{Feilforstøringsfaktor: } \left| \frac{\Delta r/r}{\epsilon} \right| = 5.0338 \cdot 10^7.$$

Husk at her er $\Delta r = -\epsilon g(20)/W'(20)$. $FE = |\Delta r|$, $RFE = FE/r = FE/20$, $BE = |\epsilon g(20)|$ $RBE = BE/|g(20)| = \epsilon = \epsilon_M = 2.2204 \times 10^{-16}$

Feilforsterkningsfaktoren:

$$EMF = RFE/RBE = 2.2355 \times 10^{-7} / (20 \times 2.2204 \times 10^{-16}) = 5.0338 \times 10^7 \approx 1/2 \times 10^8.$$

IMAx2150: Begrenset nøyaktighet



BAKGRUNNMATERIALE

Analyse detaljer for $W(20)$

La $W(x)$ være Wilkinson-polynomet, og la $\tilde{W}(x)$ være flyttalls-representasjonen av $W(x)$ der $\tilde{W}(x) = W(x) + \epsilon g(x)$.

Vi velger $r = 20$.

Vi har da at $W(r) = 0$ og $\tilde{W}(r + \Delta r) = 0$ med $\Delta r = -\epsilon g(r)/W'(r)$.

Nå er $F E = |r + \Delta r - r| = |\Delta r|$ og $B E = |W(r + \Delta r)| = |\tilde{W}(r + \Delta r) - \epsilon g(r + \Delta r)| = |\epsilon g(r + \Delta r)| = |\epsilon g(r) + \epsilon \Delta r g'(r)| \approx |\epsilon g(r)|$.

Dette gir at $R F E = |\Delta r|/|r|$ og $R B E = B E/|g(r)| = \epsilon = \epsilon_M$.

Så, forsterkningsfaktoren $f = R F E / R B E = \Delta r / (\epsilon r)$.

IMAx2150: Den deriverte av Wilkinson-polynomet

**BAKGRUNNMATERIALE**

Den deriverte av Wilkinson-polynomet:

La $W(x) = \prod_{k=1}^{k=20} (x - k)$, og la $x = r$ være en rot til polynomet.

La $W(x) = (x - r) \times f(x)$ slik at $W'(x) = f(x) + (x - r) \times f'(x)$.

Da er (*) $W'(r) = f(r) = \prod_{k=1}^{k=20} (r - k)$ for $k \neq r$.

Alternativt kan det ekvivalente til (*) vises som følger:

The derivative of $p(x) = \prod_{k=1}^n (x - \alpha_k)$ is

$$p'(\alpha_m) = \lim_{x \rightarrow \alpha_m} \frac{p(x) - 0}{x - \alpha_m} = \prod_{k \neq m} (\alpha_m - \alpha_k).$$

Da har vi også at for $j = 1, \dots, 20$ er $W'(j) = (-1)^j \times (j-1)! \times (20-j)!^1$

Eksempelvis er derfor $W'(16) = 16! \cdot 5!$ og $W'(20) = 19!$, som brukt i SAUER.

¹Denne måten å skrive $W'(r)$ på for r en rot til $W(x) = 0$ finnes i fasiten til oppgave 1.3.7 i SAUER (se s. 50 og 60?)

Bare koeffisienten -210 foran x^{19} ble endret til $-210 - 2^{-23}$ for å se på effekten på røttene til Wilkinson polynomet.

Øverst: Nøyaktige beregninger av røttene publisert i 1963 av Wilkinson.

Nederst: Beregninger av røttene gjort i 2024 med IEEE 754 DP.

Som vil vil se nedenfor, er halvparten av de opprinnelige (reelle) røttene blitt til komplekse røtter.

1.00000	2.00000	3.00000	4.00000	5.00000
6.00001	6.99970	8.00727	8.91725	20.84691
10.09527 ± 0.64350i	11.79363 ± 1.65233i	13.99236 ± 2.51883i	16.73074 ± 2.81262i	19.50244 ± 1.94033i

```

1 1.
2 2.
3 2.999999999999805
4 4.0000000000261023
5 4.999999927551538
6 6.000006943952296
7 6.999697233936014
8 8.007267603450376
9 8.917250248517071
10 20.846908101482256
11
12 10.09526233690875 - 0.6436696158377817 I
13 10.09526233690875 + 0.6436696158377817 I
14 11.793676620182865 - 1.6524927990300753 I
15 11.793676620182865 + 1.6524927990300753 I
16 13.992350040394184 - 2.5188378794049098 I
17 13.992350040394184 + 2.5188378794049098 I
18 16.730752387568366 - 2.812632592937662 I
19 16.730752387568366 + 2.812632592937662 I
20 19.502439005310507 - 1.940329388941163 I
21 19.502439005310507 + 1.940329388941163 I

```